

计算机系统基础

实验报告

Lab 0 GitLab

姓名：肖飞扬

学 号：25303050026

日期：2026 年 9 月 30 日

一、实验目的

本实验围绕 Git 与 GitHub 的基本工作流程展开。在 GitHub 建立个人实验仓库，使用 GitHub Codespaces 修改、编译和运行 C 程序，并通过提交、分支、合并和冲突处理记录完整的版本演进过程。实验的核心结果是掌握工作区、暂存区、本地仓库和远程仓库之间的关系，并能够在两个分支发生冲突时手动完成合并。

- 熟悉 `git status`、`git add`、`git commit`、`git push` 等基本命令。
- 理解暂存区的作用，并能组织内容清晰的提交。
- 创建 `feature` 分支，在 `feature` 与 `main` 分支上分别修改并提交。
- 通过修改同一行代码触发合并冲突，并完成冲突解决。
- 阅读 Git 相关资料，理解规范提交和分支管理的重要性。

二、文档问题回答

1 多人协同开发经历

没有多人协同开发的经历。

2 Git 为什么设计暂存和提交两个步骤

Git 将暂存和提交分成两个步骤，是为了让开发者有选择地组织一次提交。工作区中可能同时存在多项修改，例如程序功能、注释和文档。如果直接提交全部内容，不同目的的修改就会混在一起。`git add` 可以选择本次准备提交的文件或修改，`git commit` 再将暂存区中的内容保存为一个版本。这样能够让每次提交更集中、更容易审查，也方便问题定位和版本回退。

暂存区还提供了提交前检查的机会。使用 `git status` 可以确认哪些文件已经暂存，使用 `git diff --cached` 可以查看即将提交的具体差异，从而减少误提交。

3 `git branch` 与 `git branch -a` 的区别

`git branch` 只列出本地分支，并用星号标记当前所在分支。`git branch -a` 会列出所有分支，除了本地分支，还包括 `remotes/origin/main` 等远程跟踪分支。因此，当需要确认远程仓库中是否存在某个分支时，应使用 `git branch -a`。

三、拓展阅读与理解

1 Commit Message 规范

Commit Message 用于说明一次提交完成了什么。规范的提交信息应简洁、明确，并能反映修改类型和目的。较大型项目常在信息开头使用 `feat`、`fix`、`docs` 等类型标记，从而便于生成变更日志、查找问题和回顾项目历史。本实验中的提交信息分别说明了 `main.c` 的初次修改、`feature` 分支修改、`main` 分支修改以及冲突解决，能够直接对应提交内容。

2 Git Flow 分支控制

Git Flow 使用不同分支组织开发活动。稳定代码通常保存在主分支，新功能在独立的功能分支中开发，完成并验证后再合并回主分支。这种方式可以隔离尚未完成的修改，降低对稳定版本的影响，也允许多名开发者并行处理不同任务。本实验创建 **feature** 分支并将其合并到 **main**，是对这一思想的简化实践。

3 为什么要学习 Git

Git 能够保存文件的修改历史，使开发者了解何时、为何以及由谁进行了修改。当程序出现问题时，可以比较不同版本或恢复到之前的状态。在多人协作中，每个人可以在自己的分支上完成工作，再通过合并整合修改，避免直接覆盖他人的文件。GitHub 进一步提供远程存储、代码审查和自动化测试功能。因此，掌握 Git 不仅有助于完成课程实验，也是后续个人开发和团队协作的基础。

四、实验过程

1 建立仓库并检查状态

在 GitHub 使用模板仓库建立个人 Lab 0 仓库，并在 GitHub Codespaces 中打开项目。进入终端后，首先检查当前分支和工作区状态。

```
git switch main
git status
```

终端显示当前位于 **main** 分支，工作区最初没有未提交修改。

2 修改编译并运行 main.c

完成 **main.c** 中的 TODO，将程序输出修改为不同于原始 **Hello, world!** 的字符串。随后使用 **Makefile** 编译并运行程序。

```
make
./main
make clean
```

make 根据 **main.c** 生成可执行文件 **main**；**./main** 运行程序；**make clean** 删除目标文件和可执行文件。若 **make** 提示 **main is up to date**，说明源文件自上次编译后没有变化，现有可执行文件已经是最新版本。

3 提交 main.c 的初次修改

保存文件后，将 **main.c** 添加到暂存区并提交。

```
git add main.c
git commit -m "Update main.c"
```

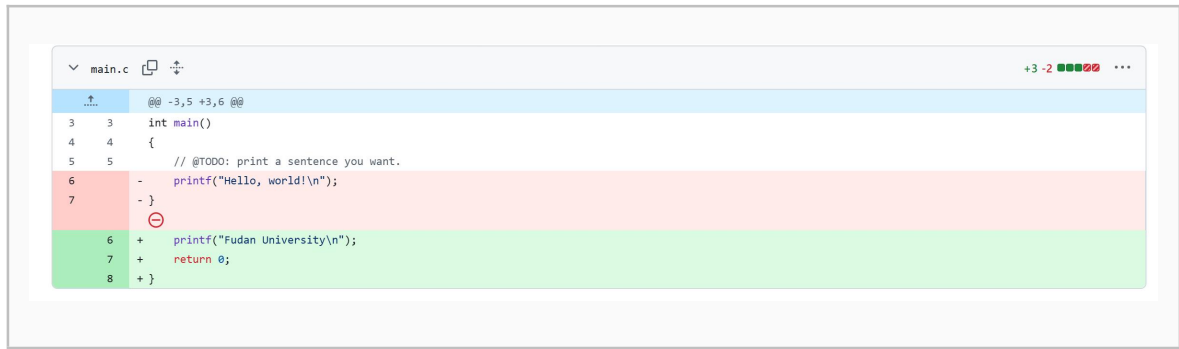


图 1 main.c 初次修改

4 创建 feature 分支并提交修改

在 main 分支已有提交的基础上，创建并切换到 feature 分支，然后修改 main.c 中的输出语句并提交。

```
git switch -c feature
# 修改并保存 main.c
git add main.c
git commit -m "Modify main.c on feature branch"
```

5 在 main 分支修改同一位置

为了确保合并时出现冲突，切回 main 分支，并对 main.c 中与 feature 分支相同的一行进行不同修改。保存后将该修改提交到 main 分支。

```
git switch main
# 对 main.c 的同一行进行不同修改并保存
git add main.c
git commit -m "Modify main.c on main branch"
```

6 合并 feature 分支并触发冲突

在 main 分支执行 git merge feature 后，Git 发现两个分支修改了 main.c 的同一位置，但修改结果不同，因此无法自动判断应保留哪一版，并报告内容冲突。

```
git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

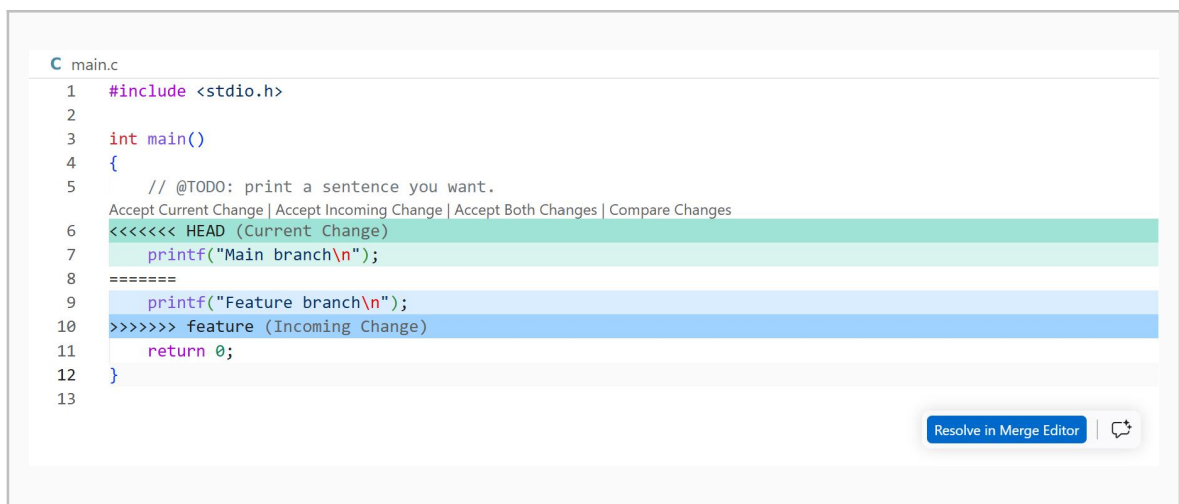


图 2 合并时出现 CONFLICT 提示

7 解决合并冲突

Git 在冲突位置加入 HEAD、分隔线和 feature 标记。HEAD 一侧表示当前 main 分支中的内容，另一侧表示 feature 分支中的内容。根据最终需要保留或重新组织代码，并删除全部冲突标记。

```
<<<<<< HEAD
printf("Main branch\n");
=====
printf("Feature branch\n");
>>>>>> feature
```

冲突处理后的文件必须只保留合法的 C 代码。例如，可以将最终输出统一为 Main and feature branches merged。保存文件后，重新将 main.c 添加到暂存区并提交，从而完成合并。

```
git add main.c
git commit -m "Resolve merge conflict"
```

```
C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Main and feature branches merged\n");
7      return 0;
8  }
9
```

图 3 删除冲突标记后的 main.c

8 检查提交历史并推送

使用图形化日志检查 main 与 feature 分支的提交关系以及最终的合并提交。确认状态正常后，将两个分支推送到 GitHub。

```
git log --oneline --all --graph --decorate
git status
git push origin main
git push -u origin feature
```

```
问题 输出 调试控制台 终端 端口
@orz258258 → /workspaces/Lab0 (main) $ git status
@orz258258 → /workspaces/Lab0 (main) $ git log --oneline --all --graph --decorate
* bed5355 (HEAD -> main) Resolve merge conflict
| \
| * e6719a3 (feature) Modify main.c on feature branch
| * | 611d000 Modify main.c on main branch
|/
* 1a7e6ee (origin/main, origin/HEAD) Update main.c
* db6f11d Initial commit
```

图 4 分支提交图与合并结果

五、实验结果与分析

本实验完成了 `main.c` 的修改、编译和运行，并使用 `Git` 记录了各阶段变化。`feature` 与 `main` 分支分别包含独立提交；两个分支修改同一行后，合并过程按预期产生冲突。通过手动选择最终内容、删除冲突标记并再次提交，`main` 分支形成了包含两条开发历史的合并结果。

关键命令及作用

命令	作用
<code>git status</code>	查看当前分支以及工作区、暂存区状态
<code>git add main.c</code>	将 <code>main.c</code> 的修改加入暂存区
<code>git commit -m</code>	把暂存区内容保存为一次本地提交
<code>git switch -c feature</code>	创建并切换到 <code>feature</code> 分支
<code>git merge feature</code>	把 <code>feature</code> 的提交历史合并到当前分支
<code>git push</code>	将本地提交上传到远程仓库